



FILE PATH TRAVERSAL

**VALIDATION OF FILE EXTENSION
WITH NULL BYTE BYPASS**

ACTIVIDAD 12

Indice

File Path Traversal: <i>Validation of file extension with null byte bypass</i>	2
Especificaciones del laboratorio.....	2
Objetivo del laboratorio	2
Marco Teórico	3
Mecanismo del filtro y su bypass mediante null byte.....	3
Clasificación según OWASP Top 10 y triada CIA.....	3
Riesgos en un entorno real.....	3
Procedimiento	4
Explicación del Payload	7
Mitigación recomendada	8
Referencias.....	8

File Path Traversal: *Validation of file extension with null byte bypass*

Especificaciones del laboratorio

Laboratorio: File Path Traversal, Traversal Sequences stripped with superfluous URL-decode

Nivel: Practitioner

Tipo de explotación: File Path Traversal, evasión de una validación de extensión de archivo mediante inyección de un byte nulo (null byte injection).

Objetivo del laboratorio

Explotar una vulnerabilidad de path traversal presente en la funcionalidad de visualización de imágenes de producto, con el fin de recuperar el contenido del archivo `/etc/passwd` del servidor. En este laboratorio la aplicación exige que el valor de filename termine con la extensión `.png` antes de aceptarlo. El objetivo específico es construir un valor que contenga una secuencia de traversal hacia `/etc/passwd` y que, al mismo tiempo, termine en la extensión esperada, de modo que la validación lo acepte sin que esa extensión llegue a formar parte de la ruta que realmente se abre en el servidor.

Marco Teórico

Mecanismo del filtro y su bypass mediante null byte

Esta variante de la vulnerabilidad exige que el parámetro filename termine con la extensión .png y cualquier extensión que no cumpla esa condición se descarta antes de intentar abrir el archivo correspondiente. PortSwigger documenta este obstáculo como una validación de extensión que puede evadirse insertando un byte nulo (%00) inmediatamente antes de la extensión que se pide (PortSwigger, s.f.). Muchos lenguajes y bibliotecas de manejo de archivos, sobre todo los que en algún punto de su implementación dependen de funciones de bajo nivel escritas en C, interpretan el byte nulo como el terminador de un string, y todo lo que aparezca después de %00 se ignora cuando se resuelve la ruta real, aunque la capa de validación de la aplicación sí evalúe el string completo, extensión incluida, antes de llegar a ese punto.

Al enviar el valor ../../etc/passwd%00.png, la validación de la aplicación revisa el string completo, comprueba que termina en .png y lo aprueba. Sin embargo, cuando ese mismo valor llega a la función del sistema de archivos que efectivamente abre el archivo, el byte nulo corta la cadena en ese punto, y lo que el sistema operativo termina abriendo es ../../etc/passwd, sin la extensión que se usó únicamente para pasar la validación.

Clasificación según OWASP Top 10 y triada CIA

Al igual que en los casos anteriores, esta vulnerabilidad corresponde a la categoría A01:2025 – Broken Access Control del OWASP Top 10 de 2025, la categoría de mayor incidencia registrada, presente en el 100% de las aplicaciones evaluadas (OWASP Foundation, 2025), y sigue mapeada a CWE-22 (MITRE Corporation, 2024). La diferencia frente a los casos previos es el tipo de defecto explotado, aquí no se trata de un patrón de traversal mal eliminado ni de un prefijo mal verificado, sino de una discrepancia entre cómo dos capas distintas del sistema interpretan el mismo string donde la capa de validación lo lee completo, mientras que la capa que abre el archivo lo trunca en el primer byte nulo.

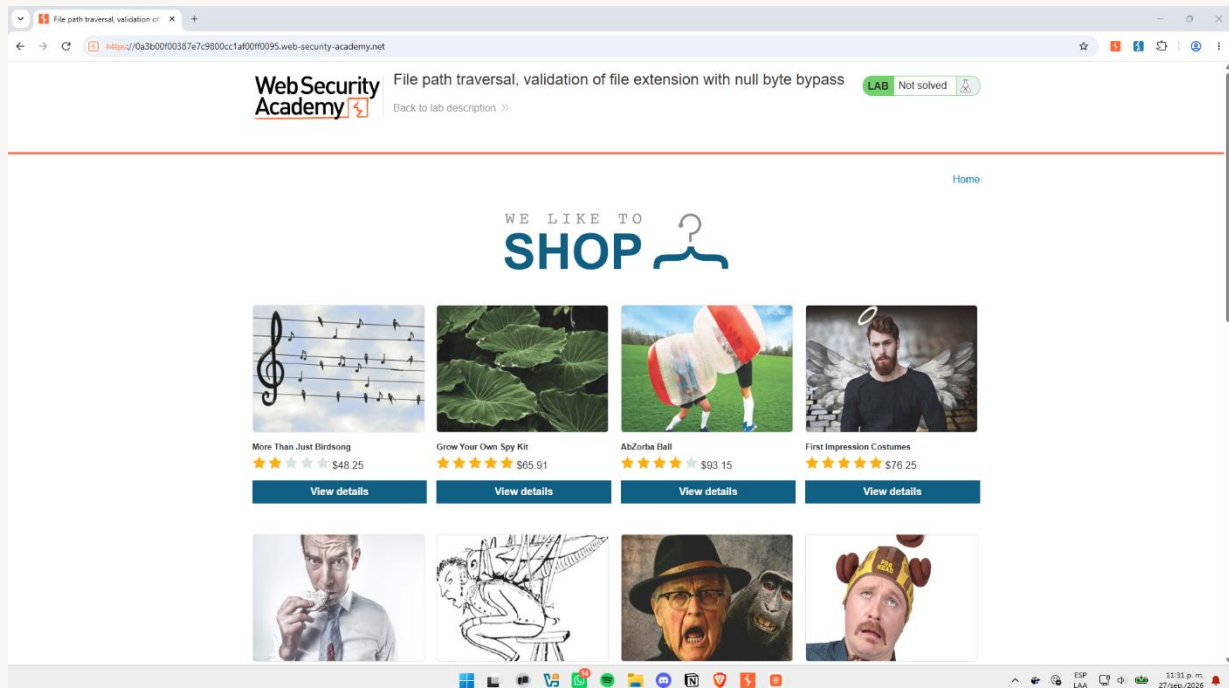
El impacto sobre la triada CIA sigue siendo el mismo que en los casos anteriores, comprometer la Confidencialidad al exponer archivos internos del servidor. Lo que distingue este caso es que expone un riesgo particular de los sistemas compuestos por varias capas o lenguajes, una validación puede ser perfectamente correcta para el string que ella misma interpreta, y aun así resultar insuficiente si otra capa del sistema interpreta ese mismo string de forma distinta.

Riesgos en un entorno real

La inyección de bytes nulos es una técnica con origen en aplicaciones construidas sobre lenguajes o bibliotecas de bajo nivel escritas en C, donde los strings terminan en un byte nulo por convención. Aunque muchos entornos de desarrollo modernos manejan los strings de forma distinta y ya no son vulnerables a esta técnica de forma directa, el riesgo persiste en aplicaciones que combinan un lenguaje de alto nivel para la lógica de negocio con bibliotecas nativas o llamadas al sistema operativo para el manejo de archivos, un punto donde la interpretación del string puede cambiar sin que el desarrollador lo note. El riesgo práctico es que una validación de extensión, aunque parezca suficiente, no garantiza que el archivo que finalmente se abre sea el que esa validación creyó estar autorizando.

Procedimiento

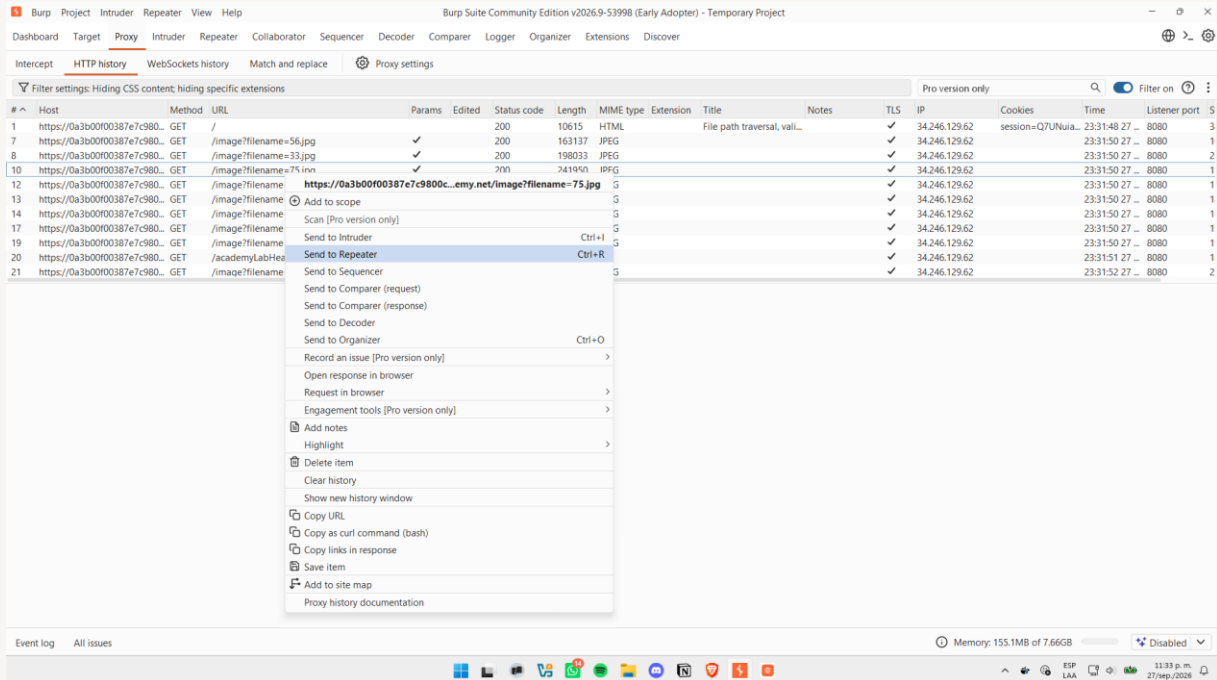
1. Se accede a la instancia del laboratorio "File path traversal, validation of file extension with null byte bypass" de PortSwigger Web Security Academy. El laboratorio inicia en estado "Not solved" y presenta una tienda en línea con productos cuyas imágenes se cargan dinámicamente.



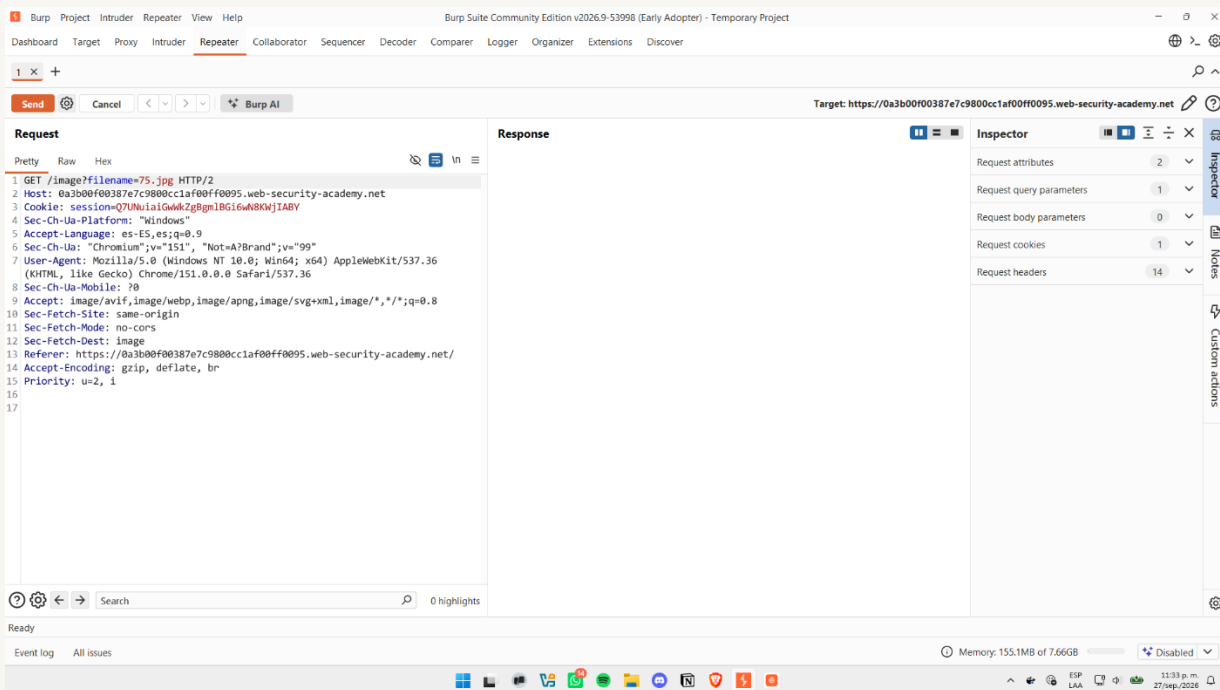
2. Con Burp Suite interceptando el tráfico, se navega por la tienda y se revisa el historial de proxy. Se confirma que cada imagen de producto se solicita mediante GET /image?filename=<archivo>.jpg.

# ^	Host	Method	URL	Params	Edited	Status code
1	https://0a3b00f00387e7c980...	GET	/			200
7	https://0a3b00f00387e7c980...	GET	/image?filename=56.jpg	✓		200
8	https://0a3b00f00387e7c980...	GET	/image?filename=33.jpg	✓		200
10	https://0a3b00f00387e7c980...	GET	/image?filename=75.jpg	✓		200
12	https://0a3b00f00387e7c980...	GET	/image?filename=43.jpg	✓		200
13	https://0a3b00f00387e7c980...	GET	/image?filename=10.jpg	✓		200
14	https://0a3b00f00387e7c980...	GET	/image?filename=16.jpg	✓		200
17	https://0a3b00f00387e7c980...	GET	/image?filename=52.jpg	✓		200
19	https://0a3b00f00387e7c980...	GET	/image?filename=34.jpg	✓		200
20	https://0a3b00f00387e7c980...	GET	/academyLabHeader			101
21	https://0a3b00f00387e7c980...	GET	/image?filename=23.jpg	✓		200

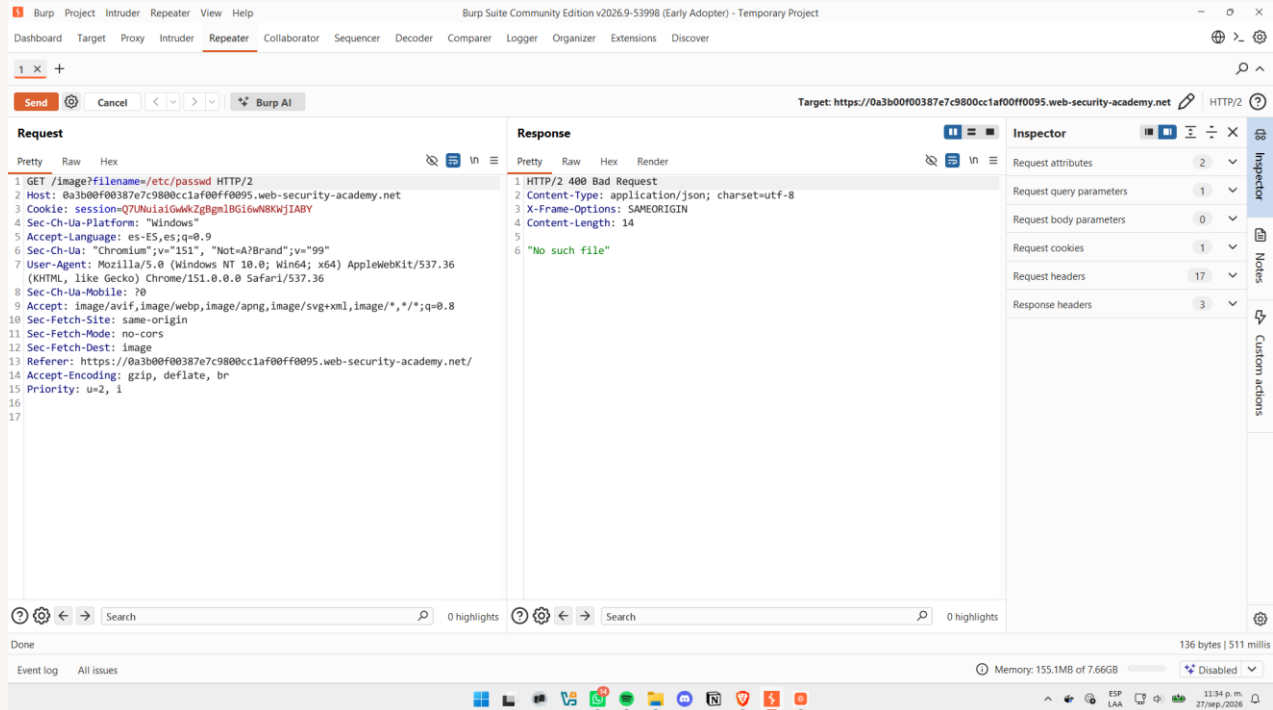
3. Se selecciona una de las solicitudes de imagen en el historial y, mediante el menú contextual, se envía al módulo Repeater para poder modificar y reenviar la solicitud de forma controlada.



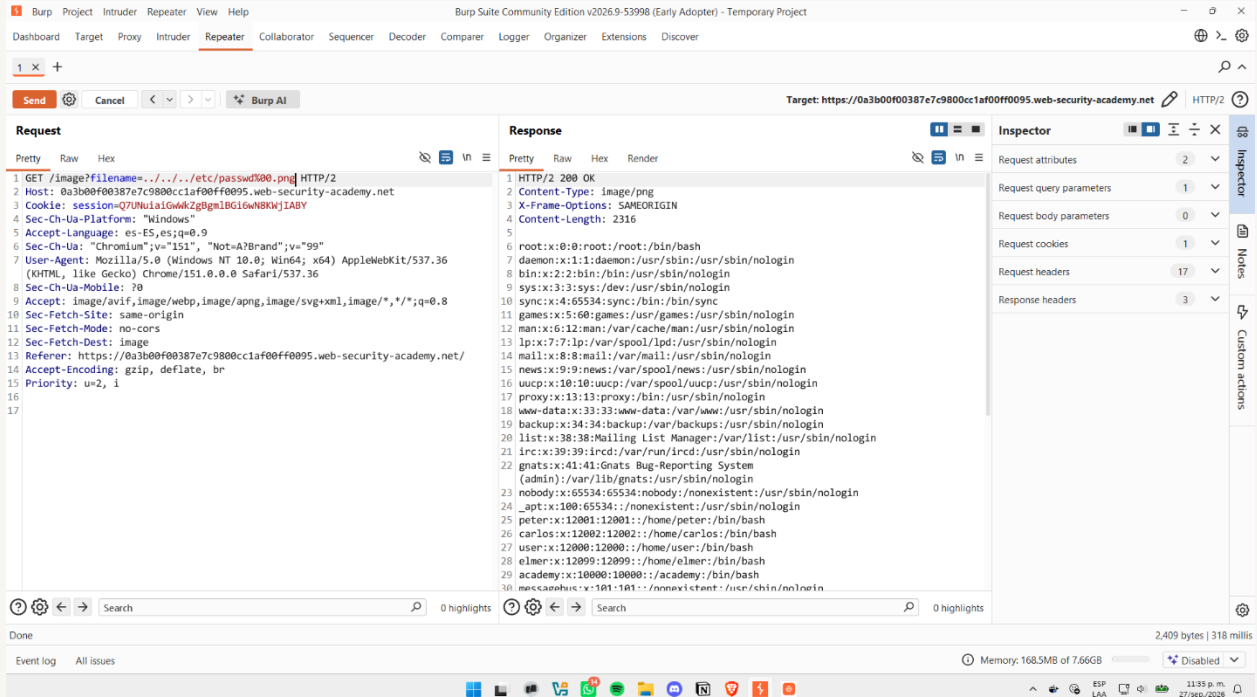
4. En Repeater se visualiza la solicitud original sin modificar: GET `/image?filename=75.jpg`, junto con las cabeceras enviadas por el navegador.



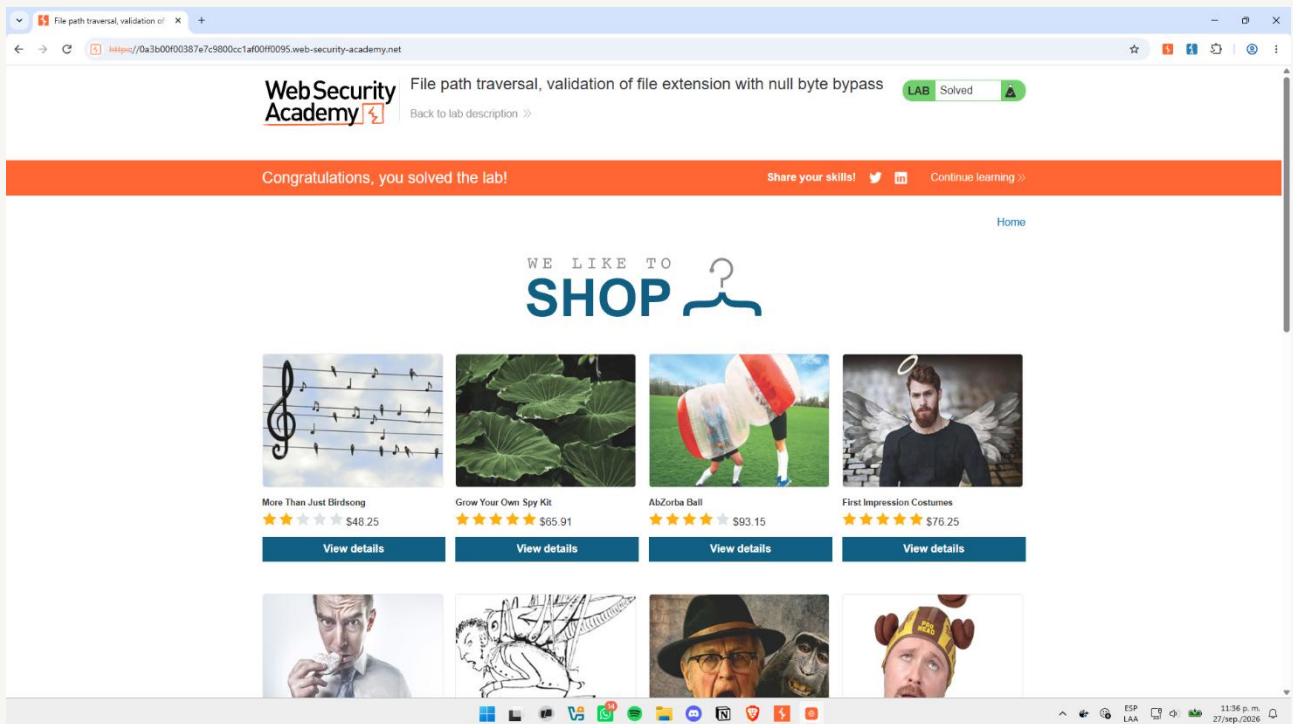
5. Se modifica filename con el valor `/etc/passwd`, sin la extensión `.png` exigida por la aplicación. El servidor responde 400 Bad Request con el mensaje "No such file", confirmando que un valor que no termina en la extensión esperada es rechazado.



6. Se modifica filename con el valor `../../etc/passwd%00.png`, donde `%00` es la codificación URL de un byte nulo insertado justo antes de la extensión `.png`. Al enviar la solicitud, el servidor responde 200 OK con Content-Type `image/png` y devuelve en el cuerpo completo del archivo `/etc/passwd`.



7. Al regresar a la aplicación, el laboratorio muestra el estado "Solved" junto con el mensaje "Congratulations, you solved the lab!", confirmando que la explotación fue exitosa.



Explicación del Payload

El payload utilizado fue `../../../../etc/passwd%00.png`, colocado como valor completo del parámetro filename. A diferencia de los casos anteriores, aquí la secuencia de traversal (`../../../../`) no es lo que se intenta ocultar, sino la extensión `.png` la que se añade deliberadamente al final, seguida inmediatamente de un byte nulo codificado como `%00` que la separa del resto de la ruta.

Su efectividad depende de que exista una diferencia entre cómo la capa de validación de la aplicación interpreta el string y cómo lo interpreta la función que finalmente abre el archivo. La validación revisa el string completo, `../../../../etc/passwd%00.png`, y confirma que termina en `.png`, por lo que lo deja pasar. La función de apertura de archivos, en cambio, trata el byte nulo como el fin de la cadena, así que en la práctica solo abre `../../../../etc/passwd`, ignorando por completo todo lo que sigue después de `%00`, incluida la extensión que sirvió para pasar la validación.

La validación de extensión de este laboratorio no tiene ningún defecto al comprobar que el string termine en `.png`, en efecto, un valor sin esa extensión es rechazado, como se confirmó en el paso 5. Donde falla es que asume que el string que la validación evalúa es exactamente el mismo que el sistema de archivos terminará usando, sin contemplar que un carácter de control como el byte nulo puede hacer que ambas interpretaciones se opongan.

Mitigación recomendada

- **Rechazar explícitamente cualquier byte nulo en el input:** antes de cualquier otra validación hay que verificar que el valor recibido no contenga el carácter nulo y descartar la solicitud de inmediato si aparece.
- **No confiar en una validación de extensión basada únicamente en el final del string (endswith):** un string puede terminar en la extensión correcta y aun así no ser la ruta que finalmente se resuelve, la extensión debe verificarse sobre la ruta ya canonicalizada, no sobre el input crudo.
- **Validación estricta por whitelist:** aceptar únicamente caracteres alfanuméricos en el nombre del archivo, rechazando cualquier valor que contenga "/", "\\ ", caracteres de control, incluido el byte nulo o secuencias de puntos, en lugar de intentar cubrir cada técnica de evasión conocida por separado.
- **Canonicalización y verificación de ruta base:** resolver la ruta completa a su forma absoluta y canónica mediante funciones nativas del lenguaje (getCanonicalPath() en Java, os.path.realpath() / Path.resolve() en Python) y verificar tanto el directorio como la extensión sobre ese resultado ya resuelto, nunca sobre el string tal como llegó del cliente.
- **Mantener actualizadas las dependencias y el entorno de ejecución:** muchos entornos modernos ya bloquean o manejan de forma segura los bytes nulos dentro de strings pasados a APIs de archivos, usar versiones actualizadas del lenguaje y sus bibliotecas reduce la superficie expuesta a esta clase de discrepancias entre capas.
- **Principio de menor privilegio:** ejecutar el proceso del servidor con permisos mínimos, de forma que, aunque un payload con byte nulo lograra evadir la validación de la aplicación, el sistema operativo continúe negando el acceso a directorios sensibles como /etc o /root.
- **Auditoría y pruebas específicas de bypass:** al probar validaciones de extensión de archivo, incluir explícitamente payloads con bytes nulos codificados, además de las variantes ya cubiertas anteriormente (rutas absolutas, secuencias anidadas, doble codificación URL, prefijos de directorio).

Referencias

MITRE Corporation. (2024). CWE-22: Improper limitation of a pathname to a restricted directory ('Path Traversal'). Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>

OWASP Foundation. (2025). A01:2025 – Broken Access Control. OWASP Top 10:2025. https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/

PortSwigger. (s. f.). Lab: File path traversal, validation of file extension with null byte bypass. Web Security Academy. <https://portswigger.net/web-security/file-path-traversal/lab-validate-file-extension-null-byte-bypass>

PortSwigger. (s. f.). What is path traversal, and how to prevent it? Web Security Academy. <https://portswigger.net/web-security/file-path-traversal>